Architecture and Protocol

Browser-Side Recall and Recovery

The browser maintains three related data structures: a segments map containing the latest accepted revision of each segment, a rolling finals list for recall, and a rows map connecting segments to their rendered DOM elements.

To support “What did I miss?”, the client retains the latest 65 seconds of finalized captions. We selected 65 seconds rather than exactly 60 seconds to provide a small margin for timestamp rounding and uneven caption intervals. After each accepted caption, the client calculates cutoff = latest source timestamp − 65 seconds.

Finalized captions older than the cutoff are removed from the rolling list, segment map, and displayed transcript. Provisional captions are not removed by this process because they may still receive newer revisions. The resume high-water mark, lastFinalSegment and lastFinalRevision, is not reduced when old local history is pruned. Therefore, reconnecting clients do not incorrectly request already-processed older segments.

The recall interface supports 15, 30, and 60-second windows, with 30 seconds selected by default. Recall is normally performed entirely from the local client state. The client filters only finalized captions whose source_ts falls inside the selected window and orders them from oldest to newest. No WebSocket request is generated when the user changes the recall window. The interface also reports when the client has insufficient local history, such as shortly after a late join.

Reconnecting to the server

To prevent overloading the server during recovery, the client implements exponential backoff with jitter when scheduling reconnection attempts. Without randomized delays, a server outage or network interruption would cause all disconnected clients to retry simultaneously. In scheduleReconnect(), the base backoff scales exponentially (RECONNECT_MIN_MS * 2^exp, capped at an exponent of 10 and bounded by RECONNECT_MAX_MS) and is multiplied by a random jitter factor between 0.75 and 1.25 (± 25%). This distributes reconnection requests evenly over time, protecting the server from traffic spikes as it comes back online.

Revision and Recovery Semantics

The Server maintains a dict containing the most recent revisions of each segment, as well as whether each segment is final or not. Whenever the server receives a new CAPTION_UPDATE from the source, it drops it if:

- A more recent revision of the same segment has already been stored
- The segment has already been marked as final

Whenever the server receives a CLIENT_HELLO, it will reply with a HISTORY frame containing only finalized captions newer than the client's last_segment/last_revision, within history_seconds (capped at 60), oldest first.

On the client side, all live and recovered captions pass through the same applyCaption() function, preventing HISTORY recovery from bypassing the normal revision and deduplication rules. The client keeps one row per segment on screen, so a newer revision simply replaces the older text in place. Once a segment is marked as final, it can never be overwritten, and any older (stale) or repeated (duplicate) revisions that arrive late are ignored.

On every initial connection or reconnect, the client sends the most recent finalized segment and revision in CLIENT_HELLO and requests 60 seconds of recovery history. Requesting 60 seconds upfront ensures that all recall-window options (15, 30, and 60 seconds) can be populated locally after a late join or disconnection without needing another server request when the user switches views.

When a HISTORY message arrives, the client validates that the response contains an array, parses each caption using the same parser used for live updates, and sorts valid items from oldest to newest before passing each item through applyCaption(caption, true) to fill in any missing gaps without creating duplicates. Malformed, recovered, duplicate, and stale items are recorded in client-side metrics. Although the client cleans up visible finalized captions after 65 seconds to save memory, it remembers which segments were already finalized in a finaled deduplication map for 180 seconds. Since this is longer than the server's 60-second history window, an old final caption that was already cleaned up from the screen can never be added again after a reconnect.

Backpressure and Overload Policy

Each client has its own bounded queue and sender task, so a slow client only delays itself. While a client keeps up, it receives every revision. Once a queued message has waited over 1s, a newer revision replaces older unsent revisions of the same segment, and finals are never replaced. TCP's buffers originally hid slow clients, so the server now also pings each client regularly. If a pong arrives more than 250ms later than usual, we stop writing and let the backlog collect on the server, where old revisions are discarded. We then measure the client's real read rate, pace it below that rate, and send its waiting finals together in one HISTORY message. If its queue still holds 2048 messages after dropping all old revisions and provisionals, we disconnect it so it can reconnect and catch up through HISTORY.

Public Experiments

Please refer to the Appendix for exact configuration and commands.

Metric	Test 1 Median	Test 2 Median	Test 3 Median
P50 Caption Age (ms)	1.92	4.13	4.02
P95 Caption Age (ms)	3.22	6.69	9.15
P99 Caption Age (ms)	4.07	7.82	3 326.27

Fresh Caption Ratio (%)	100	100	97.73
Final Caption Completeness (%)	100	100	95.26
Final completeness (%)	100	100	100
Normal Update Coverage (%)	100	100	94.69
Normal-client P95 (ms)	3.22	6.69	6.35
Slow-client P95 (ms)	Not applicable	Not applicable	7 353.21

T4 - Reconnect

Metric	Run 1	Run 2	Run 3	Median
Recovered HISTORY Items	120	120	140	120
Recovery P50 (ms)	3.63	2.58	12.55	3.63
Recovery P95 (ms)	4.03	8.36	13.76	8.36
Successful Connections	120	120	120	120
Duplicate Messages	0	0	0	0
Stale Messages	0	0	0	0
Connection Errors	0	0	0	0
Final Caption Completeness (%)	100	100	100	100

*See appendix for exact commands and raw figures for T4

T5 - Network Impairment

Median comparison

Metric	Baseline	Jitter	Fixed Delay	Packet Loss
P50 Caption Age (ms)	11.02	209.05	218.10	10.79
P95 Caption Age (ms)	21.77	252.90	226.78	210.77
P99 Caption Age (ms)	28.66	272.14	250.22	235.43
Fresh Caption Ratio (%)	100.00	100.00	100.00	99.96
Fresh Final Caption Ratio (%)	100.00	100.00	100.00	99.84
Normal Update Coverage (%)	100.00	100.00	100.00	99.91
Final Caption Completeness (%)	100.00	100.00	100.00	100.00
Duplicate Messages	0	0	0	0
Stale Messages	0	0	0	0
Connection Errors	0	0	0	0

*See appendix for exact commands and raw figures for T5

Optimizations

The 2 main optimizations we implemented are:
1. Flow control: monitors the responsiveness of each client and adjusts the sending rate for clients that exhibit signs of lag / slow clients. It works via ping probing to detect if the client experiences a sudden burst of lag if RTT exceeds the baseline by some threshold, and it also acts as a mechanism to estimate how much data can reach the client based on the time taken between pong responses and the amount of data that was sent between their corresponding pings
2. Batch History: bundles multiple backlog updates into a single HISTORY frame when a client is severely overloaded and in a degraded state, significantly reducing framing overhead and maximizing data transfer when the sending rate is restricted.

In order to really display the difference between optimizations we set harsher network conditions and run a slow-client pool with disconnects. The exact configuration can be found in the appendix.
The following are the results:

Run (median of 3 runs each)	**Baseline**	**Flow Control**	**Flow control + Batch history**
P50 Caption Age (ms)	1,491.19	435.45	420.59
P95 Caption Age (ms)	5,382.45	41,971.17	2,052.73
P99 Caption Age (ms)	31,719.20	72,249.29	8,127.93
Fresh Caption Ratio (%)	1.07	52.87	55.52
Fresh Final Caption Ratio (%)	0.63	20.51	23.92
Final Caption Completeness (%)	89.89	59.55	95.21
Normal Update Coverage (%)	81.46	39.57	25.99
Normal Clients P95 (ms)	2,991.05	36,415.81	1,692.01
Slow Clients P95 (ms)	60,224.84	73,799.08	13,628.94
Recovery Clients P95 (ms)	1,996.92	801.31	86,483.06

We observe that under harsh network conditions, flow control measures are significantly stricter to allow for more message coalescing where we drop provisional revisions to reduce the amount we send. This improves the message freshness significantly, however it also causes some final caption updates to fail to reach the client, dropping the final caption completeness.

History batching works in tandem with flow control to mitigate this. When the client is significantly degraded, we batch multiple caption updates into a single history frame instead, significantly reducing the overhead in sending and allowing the client to catch up in final captions.

Recovery P95 for the batching column is a measurement artifact. The analyser counts every HISTORY frame as recovery, timed from connect, including batched finals sent during the live stream. Measuring only the first HISTORY after each reconnect gives about 1s.

Limitations

As we are using the difference in RTT of the ping packets to estimate the amount of data that the client is able to process at any given time, information regarding the clients rate of receiving is therefore delayed. Sending pace may be slow to adjust to changes in network connections. Furthermore, we do not consider the rate at which client-side processes these packets. The assumption is that the client is able to process and display caption updates quicker than it receives them.

Failures

If a client is too slow and its queue still holds MAX_PENDING_MESSAGES (2048) messages after we drop all old revisions and provisionals, we close the connection (code 1013). The client then reconnects automatically and catches up through HISTORY. Because HISTORY covers at most the last 60 seconds, a client that takes longer than 60 seconds to reconnect loses any finals older than that.

Trade-offs

There is a noticeable tradeoff between the “freshness” of messages received by the clients vs update coverage of the captions received. This is because if we want to maintain high “freshness” under severe network constraints, we must drop packets that are too old (i.e. not fresh) to prevent stale messages from accumulating in the queue. This inevitably means that not all revisions will be received by the clients. On the other hand, if we want to increase the number of revisions seen by the client, we must preserve stale “unfresh” messages in our client’s queue. This harms the freshness measure but it will guarantee that clients see a more complete set of revisions, improving coverage.

Future-Work

The current architecture only supports having a single source connected at a time. A potential future implementation could support multiple sources, multiple sessions within the same source, or multiple sources with the same session. This would involve implementing per-source bounded histories and proper synchronisations mechanisms.

Another direction to take this is to implement visual playback as well, such that students can keep up with lecture annotations alongside its captions.

Finally, the system could be scaled by deploying multiple server instances in Kubernetes for redundancy and load balancing across a large enterprise or university. Since the server currently stores the 60-second revision dictionary in local memory, running multiple pods would require moving the bounded history to a shared store like Redis and using a pub/sub broker to broadcast live CAPTION_UPDATE messages across all pods. This would allow clients to reconnect to any available pod if an instance fails without losing session history.

Appendix

T1 commands

```python
python run_demo.py --clients 20 --duration 20 --runs 3 --output-stem t1
```

T2 commands

```python
python run_demo.py --clients 100 --duration 40 --runs 3 --output-stem t2
```

T3 commands

```python
python run_demo.py --clients 100 --duration 60 --runs 3 \
  --slow-percent 10 --slow-ms 500 --output-stem t3
```

T4 commands and results

Commands

```python
python run_demo.py --clients 100 --duration 60 --runs 3 \
  --disconnect-percent 20 --disconnect-at 15 \
  --disconnect-seconds 5 --output-stem t4
```

Additional Median Results

Metric	Median
P50 Caption Age (ms)	11.18
P95 Caption Age (ms)	26.27
P99 Caption Age (ms)	32.68
Fresh Caption Ratio (%)	100.00
Fresh Final Caption Ratio (%)	98.68
Normal Update Coverage (%)	98.33

T5 commands and results

Baseline configuration commands

```python
sudo ./network_profiles/reset.sh lo

python3 run_demo.py --clients 100 --duration 40 \
  --runs 3 --output-stem t5_baseline
```

No network impairment was applied.

Baseline results

Metric	Run 1	Run 2	Run 3	Median
P50 Caption Age (ms)	11.62	11.01	11.02	11.02
P95 Caption Age (ms)	37.11	21.77	21.37	21.77
P99 Caption Age (ms)	98.81	28.66	27.21	28.66
Fresh Caption Ratio (%)	100.00	100.00	100.00	100.00
Fresh Final Caption Ratio (%)	100.00	100.00	100.00	100.00
Normal Update Coverage (%)	100.00	100.00	100.00	100.00
Final Caption Completeness (%)	100.00	100.00	100.00	100.00
Duplicate Messages	0	0	0	0
Stale Messages	0	0	0	0

Fixed-delay configuration commands

```python
sudo ./network_profiles/reset.sh lo
sudo tc qdisc replace dev lo root netem delay 100ms

python3 run_demo.py --clients 100 --duration 40 \
  --runs 3 --output-stem t5_delay

sudo ./network_profiles/reset.sh lo
```

Impairment applied: a constant 100 ms delay without additional jitter or packet loss.

Fixed-delay results

Metric	Run 1	Run 2	Run 3	Median
P50 Caption Age (ms)	218.10	218.29	215.41	218.10
P95 Caption Age (ms)	228.78	226.78	225.92	226.78
P99 Caption Age (ms)	266.63	250.22	238.55	250.22

Fresh Caption Ratio (%)	100.00	100.00	100.00	100.00
Fresh Final Caption Ratio (%)	100.00	100.00	100.00	100.00
Normal Update Coverage (%)	100.00	100.00	100.00	100.00
Final Caption Completeness (%)	100.00	100.00	100.00	100.00
Duplicate Messages	0	0	0	0
Stale Messages	0	0	0	0

Jitter commands and results

Configuration commands

```python
sudo ./network_profiles/delay_100ms.sh lo

python3 run_demo.py --clients 100 --duration 40 \
  --runs 3 --output-stem t5_jitter

sudo ./network_profiles/reset.sh lo
```

Impairment applied: approximately 100 ms mean delay with normally distributed 20 ms jitter.

Jitter results

Metric	Run 1	Run 2	Run 3	Median
P50 Caption Age (ms)	207.82	209.05	209.06	209.05
P95 Caption Age (ms)	253.72	251.93	252.90	252.90
P99 Caption Age (ms)	274.38	271.66	272.14	272.14
Fresh Caption Ratio (%)	100.00	100.00	100.00	100.00
Fresh Final Caption Ratio (%)	100.00	100.00	100.00	100.00
Normal Update Coverage (%)	100.00	100.00	100.00	100.00
Final Caption Completeness (%)	100.00	100.00	100.00	100.00
Duplicate Messages	0	0	0	0
Stale Messages	0	0	0	0

Packet-loss commands and results

Configuration commands

```python
sudo ./network_profiles/loss_3pct.sh lo
```

```
python3 run_demo.py --clients 100 --duration 40 \
  --runs 3 --output-stem t5_loss

sudo ./network_profiles/reset.sh lo
```

Impairment applied: 3% IP packet loss without additional delay or jitter.

Packet-loss results

Metric	Run 1	Run 2	Run 3	Median
P50 Caption Age (ms)	10.79	11.20	10.46	10.79
P95 Caption Age (ms)	209.98	210.77	212.85	210.77
P99 Caption Age (ms)	273.65	235.07	235.43	235.43
Fresh Caption Ratio (%)	99.96	99.98	99.39	99.96
Fresh Final Caption Ratio (%)	99.84	99.96	98.14	99.84
Normal Update Coverage (%)	99.54	99.91	99.93	99.91
Final Caption Completeness (%)	100.00	100.00	100.00	100.00
Duplicate Messages	0	0	0	0
Stale Messages	0	0	0	0

Optimizations Data

Network configuration

```shell
sudo tc qdisc del dev "$IFACE" root 2>/dev/null || true
sudo tc qdisc add dev "$IFACE" root handle 1: tbf rate 1mbit burst 32kbit latency 1000ms
sudo tc qdisc add dev "$IFACE" parent 1:1 handle 10: netem delay 80ms 30ms loss 4%
```

Test configurations

```python
python run_demo.py --clients 200 --duration 120 --runs 3 --slow-percent 15 --slow-ms 750
--disconnect-percent 25 --disconnect-at 20 --disconnect-seconds 8
```

Baseline (LIVERECALL_FLOW_CONTROL=0)

run	1	2	3	Median
P50 Caption Age (ms)	1,405.43	1,491.19	1,544.53	1,491.19
P95 Caption Age (ms)	5,382.45	5,074.18	5,802.05	5,382.45
P99 Caption Age (ms)	31,719.20	28,189.49	45,181.08	31,719.20

Fresh Caption Ratio (%)	3.43	0.64	1.07	1.07
Fresh Final Caption Ratio (%)	2.43	0.06	0.63	0.63
Final Caption Completeness (%)	89.89	90.30	88.31	89.89
Normal Update Coverage (%)	81.50	81.46	81.29	81.46
Normal Clients P95 (ms)	2,917.38	2,991.05	3,074.08	2,991.05
Slow Clients P95 (ms)	60,224.84	46,345.62	65,930.67	60,224.84
Recovery Clients P95 (ms)	1,849.06	2,768.27	1,996.92	1,996.92

Flow control (LIVERECALL_BATCH_FINALS=0)

run	1.00	2.00	3.00	Median
P50 Caption Age (ms)	435.45	452.92	392.03	435.45
P95 Caption Age (ms)	36,980.82	41,971.17	44,601.04	41,971.17
P99 Caption Age (ms)	70,986.12	72,506.54	72,249.29	72,249.29
Fresh Caption Ratio (%)	52.87	52.17	56.16	52.87
Fresh Final Caption Ratio (%)	20.22	20.51	21.17	20.51
Final Caption Completeness (%)	58.70	59.55	59.72	59.55
Normal Update Coverage (%)	39.57	40.03	39.02	39.57
Normal Clients P95 (ms)	29,386.67	36,415.81	37,755.64	36,415.81
Slow Clients P95 (ms)	68,056.09	73,799.08	78,404.22	73,799.08
Recovery Clients P95 (ms)	602.44	801.31	1,129.32	801.31

Flow control + History batching (Default settings)

run	1.00	2.00	3.00	Median
P50 Caption Age (ms)	420.59	418.02	448.68	420.59
P95 Caption Age (ms)	5,506.92	1,868.47	2,052.73	2,052.73
P99 Caption Age (ms)	12,007.53	7,620.80	8,127.93	8,127.93
Fresh Caption Ratio (%)	55.52	56.47	54.07	55.52
Fresh Final Caption Ratio (%)	23.92	23.93	23.72	23.92
Final Caption Completeness (%)	95.04	95.45	95.21	95.21
Normal Update Coverage (%)	25.28	26.03	25.99	25.99
Normal Clients P95 (ms)	2,578.44	1,504.63	1,692.01	1,692.01
Slow Clients P95 (ms)	13,628.94	13,691.67	13,206.39	13,628.94
Recovery Clients P95 (ms)	86,483.06	86,009.16	87,961.24	86,483.06